

EAGLE EYE
[AUTOMATIC LICENSE NUMBER PLATE DETECTION AND
RECOGNITION]

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
21010301243	21013001656 of 2021-2022	BWU/BCA/21/264	ARIJIT GUPTA
21010301244	21013001657 of 2021-2022	BWU/BCA/21/265	RISHOV NANDI
21010301232	21013001643 of 2021-2022	BWU/BCA/21/250	SAYAN JASH
21010301246	21013001659 of 2021-2022	BWU/BCA/21/267	NEEL SHARMA
21010301229	21013001640 of 2021-2022	BWU/BCA/21/247	SHUBHAJIT DAS

*in partial fulfilment for the award of the degree
of*

BACHELOR OF COMPUTER APPLICATION



Department of Computational Sciences

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

July 2024

EAGLE EYE
**[AUTOMATIC LICENSE NUMBER PLATE DETECTION AND
RECOGNITION]**

Submitted by

Roll Number	Registration Number	Student Code	Student Name
21010301243	21013001656 of 2021-2022	BWU/BCA/21/264	ARIJIT GUPTA
21010301244	21013001657 of 2021-2022	BWU/BCA/21/265	RISHOV NANDI
21010301232	21013001643 of 2021-2022	BWU/BCA/21/250	SAYAN JASH
21010301246	21013001659 of 2021-2022	BWU/BCA/21/267	NEEL SHARMA
21010301229	21013001640 of 2021-2022	BWU/BCA/21/247	SHUBHAJIT DAS

in partial fulfilment for the award of the degree

of

BACHELOR OF COMPUTER APPLICATION

in

COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

COMPUTATIONAL SCIENCES

BONAFIDE CERTIFICATE

Certified that this project report “**EAGLE EYE [Automatic License Number Plate Detection and Recognition]**” is the bonafide work of “**ARIJIT GUPTA, RISHOV NANDI, SAYAN JASH, NEEL SHARMA, SHUBHAJIT DAS**” who carried out the project work under my supervision.

SIGNATURE

Dr. Jayanta Aich
HEAD OF THE DEPARTMENT

Computational Sciences
*398, Ramkrishnapur Road, Barasat,
North 24 Parganas, Kolkata - 700 125*

SIGNATURE

Dr. Rahul Kumar Ghosh
SUPERVISOR

Assistant Professor
Computational Sciences

External Examiner

ABSTRACT

EAGLE EYE (Automatic License Number Plate Detection and Recognition) systems have gained significant traction in recent years due to their wide-ranging applications in traffic management, law enforcement, and security. This project explores the design, implementation, and evaluation of an ALPD system aimed at enhancing vehicle identification and monitoring capabilities.

The project begins with a comprehensive analysis of existing ALPD techniques and technologies, focusing on image processing, pattern recognition, and machine learning algorithms suitable for real-time application. A prototype ALPD system is developed using python, OpenCV, Teseract OCR.

Key components of the system include preprocessing modules for image enhancement, segmentation algorithms for isolating number plates, character recognition using deep learning models, and integration with a database for storing and retrieving vehicle information. The system is designed to operate efficiently under varying lighting conditions and vehicle speeds, ensuring robust performance in practical scenarios.

Evaluation of the developed ALPD system involves testing with real-world datasets and benchmarking against established metrics such as accuracy, speed of detection, and robustness to environmental factors.

The implications of this project extend to improved traffic management, enhanced law enforcement capabilities, and increased security in public and private sectors. Future work includes scalability enhancements, integration with smart city initiatives, and further optimization for deployment in diverse operational environments.

ACKNOWLEDGEMENT

We would like to express our deepest gratitude to all those who have contributed to the successful completion of this EAGLE EYE project.

First and foremost, we are immensely thankful to Dr. Rahul Kumar Ghosh, for their invaluable guidance, constant support, and expert advice throughout the duration of this project. Their insights and feedback were instrumental in shaping the direction and execution of our research.

We are grateful to the faculty members of the Computational Sciences at Brainware University, whose lectures, discussions, and encouragement provided a solid foundation for understanding the theoretical concepts and practical aspects of image processing and machine learning.

Special thanks go to our project team members, Arijit Gupta, Rishov Nandi, Neel Sharma, Sayan Jash, Shubhajit Das, for their collaborative effort, dedication, and teamwork. Each member's unique skills and contributions played a crucial role in the development and implementation phases of the ALPD system.

Lastly, we extend my heartfelt thanks to our family and friends for their unwavering support, patience, and encouragement throughout this endeavor.

This project would not have been possible without the support and contributions of these individuals and organizations. Their assistance and encouragement have been invaluable, and we are deeply appreciative of their involvement.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO.
	Abstract	<i>iii</i>
	Acknowledgement	<i>iv</i>
	List of Figures	<i>vi</i>
1	Introduction	1-2
	1.1 Use Cases	2
	1.2 Project Architecture	2
2	Labelling	3-8
	2.1 Understand and Collect required Data	3
	2.2 Label Images	3
	2.3 Parsing Information from XML	4
	2.4 Parsing Data from XML and Converting it into CSV	5
	2.5 Verify Data	7
3	Data Processing	8-9
	3.1 Read Data	8
	3.2 Split, Train and Test Set	9
4	Deep Learning for Object Detection	9-15
	4.1 Model Building	9
	4.2 Training and Save	14
5	Pipeline Object Detection Model	15-19
	5.1 Make Predictions	15
	5.2 De-Normalize the Output	16
	5.3 Bounding Box	17
	5.4 Create Pipeline	18
6	Optical Character Recognition - OCR	20-21
	6.1 Tesseract OCR	20
	6.2 Limitation of Pytesseract	20
	6.3 Extract Number Plate Text from Image	20
7	Real Time Number Plate Recognition With Yolo	21-40
	7.1 Explanation of Required Data	21
	7.2 Data Preparation	25
	7.3 Training YOLO	29
	7.4 Predicting Results from YOLO	35
	7.5 Real Time Object Detection	39
8	Conclusion	40
9	References	41

List of Figures

SL NO.	TOPIC	PAGE NO.
1	<i>Figure 1.1-Detecting Number Plates</i>	1
2	<i>Figure 1.2 – Implementation of Number Plate Detector</i>	2
3	<i>Figure 1.3 – Framework</i>	2
4	<i>Figure 1.4 – Number Plate Recognition Project Architecture</i>	3
5	<i>Figure 2.1 Sample 1</i>	3
6	<i>Figure 2.2 Sample 2</i>	3
7	<i>Figure 2.3 Labelling</i>	4
8	<i>Figure 2.4 – N2.jpeg with bounding box</i>	8
9	<i>Figure 4.1 - Diagram</i>	10
10	<i>Figure 5.1 – Test Image</i>	16
11	<i>Figure 5.2 – Detecting Number Plate</i>	18
12	<i>Figure 5.3 – Detecting Number Plate after Pipeline</i>	19
13	<i>Figure 6.1 – Cropped Image</i>	20
14	<i>Figure 7.1 – Procedure Diagram</i>	22
15	<i>Figure 7.2 – Measuring width and height of number plate</i>	23
16	<i>Figure 7.3 – Diagram of process of training and testing</i>	23
17	<i>Figure 7.4 – Various Images of Real Time Number Plate Detection</i>	35
18	<i>Figure 7.5 – Test Image</i>	36
19	<i>Figure 7.6 – Detecting Number Plate</i>	39
20	<i>Figure 7.7 – Real Time Number Plate Detection Using YoloV5</i>	40

EAGLE EYE

**(AUTOMATIC LICENSE NUMBER PLATE DETECTION AND
RECOGNITION)**



1. INTRODUCTION



Figure 1.1-Detecting Number Plates

Security has emerged as one of the top issues facing every firm in the modern day, and automation of this kind of protection is crucial. Nonetheless, a lot of the existing solutions—which frequently rely on numerous constraints—are still not reliable in practical settings. In the subsequent

Today, we will learn how to use the Python programming language to recognize license plate numbers. For this project, we will use Python Pytesseract to extract the characters and numbers from the license plate and OpenCV to recognize the license number plates.

Additionally, this research will demonstrate a reliable and effective ALPR system built on the cutting-edge Yolo object detector. By the end of the Web application, we'll have a Python script that can identify the license plate automatically.

1.1 USE CASES

Finding the area of the vehicle that is most likely to house the license plate is known as license plate detection. The process of recognizing the values on a license plate is called recognition. The technology that employs computer vision to identify and locate a license plate from an input image of an automobile is known as license plate detection and recognition. This technique has numerous applications. It is used to identify vehicles that are violating traffic laws when they are on the road. It is used in security to take down the license plates of cars entering and leaving specific locations. It is employed to take down the license plates of vehicles that are being parked in parking lots. There is an endless range of applications for it.



Figure 1.2 – Implementation of Number Plate Detector

1.2 PROJECT ARCHITECTURE

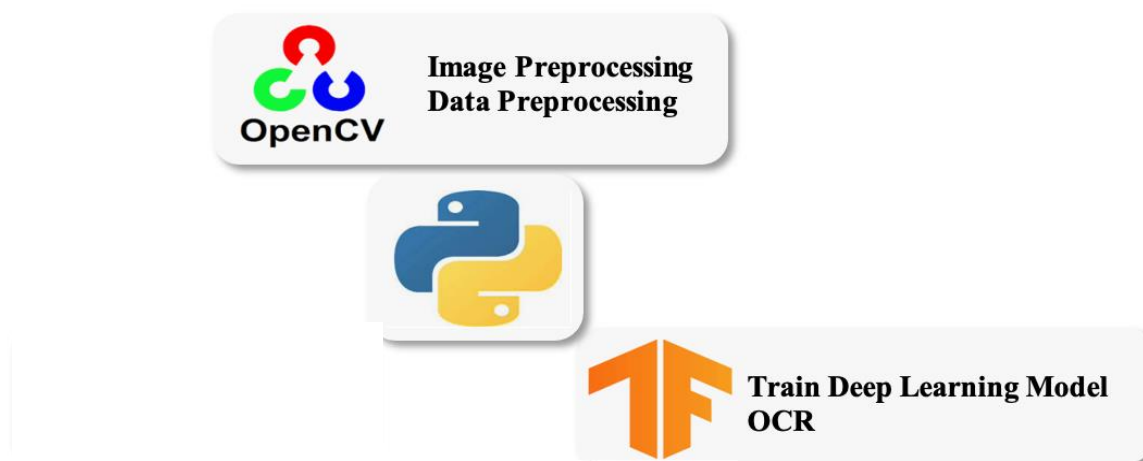


Figure 1.3 - Framework

Six modules make up the architecture shown in Figure 2 below. RESTful API, Labeling, Training, Save Model, OCR, and Pipeline. This is the procedure. We shall first gather the picture. The next step is to use the open-source, Python GUI-developed Image Annotation Tool Github to label photos for object detection of license plates or number plates. Following the image's labeling, we will focus on data preprocessing before creating and honing a TensorFlow 2 deep learning object identification model called Inception Resnet V2. Following the completion of the training phase for the Object Detection model, we will use this model to trim the picture containing the license plate, also known as the region of interest (ROI), and send the ROI to the optical.

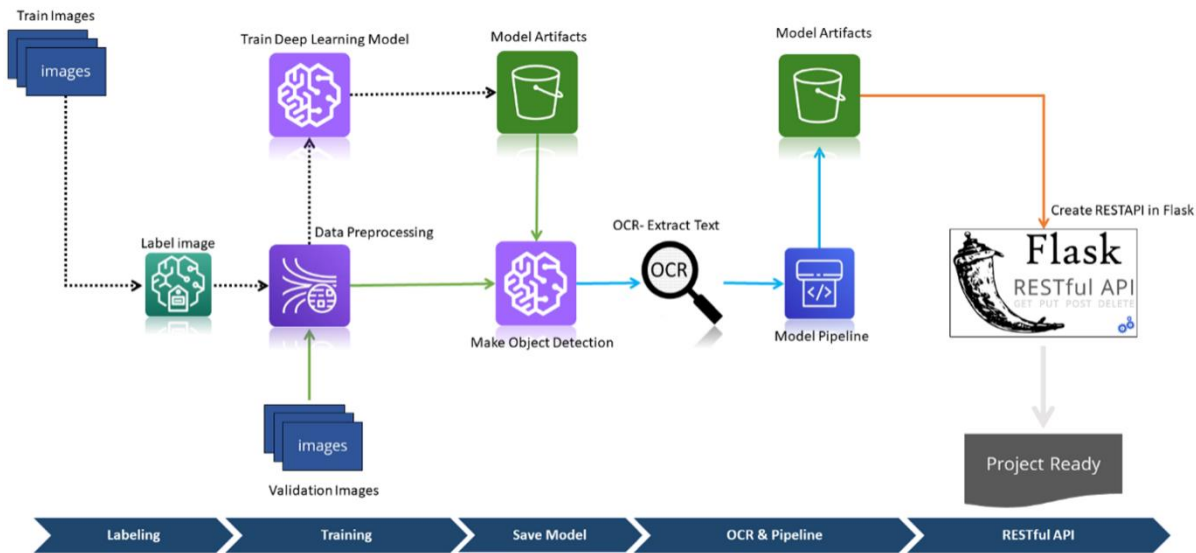


Figure 1.4 – Number Plate Recognition Project Architecture

2. LABELING

2.1 UNDERSTAND & COLLECT REQUIRED DATA

We require data in order to construct the license plate recognition system. In order to do that, we must gather as many pictures of vehicles as we can that have the license plate visible on them. Here is the sample data that I used for building this project. A few examples are shown below. Figures 2.1 and 2.2



Figure 2.1 Sample 1



Figure 2.2 Sample 2

2.2 LABEL IMAGES

I utilized the Labelling image annotation tool for label images. To install the package, download the labellmg from GitHub and follow the installation instructions. Following that, launch the GUI, follow the instructions, select CreateRectBox, design the rectangular box as seen below, and save the result in XML format.

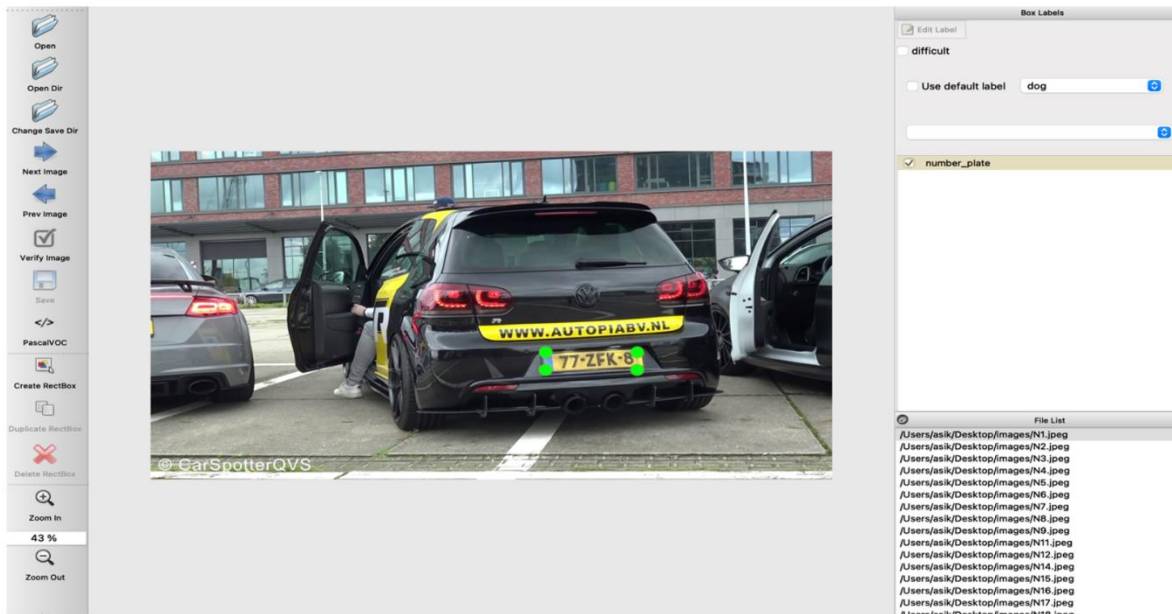


Figure 2.3 Labelling

2.3 PARSING INFORMATION FROM XML

Example of our .xml files will look as below.

```
<annotation>

  <folder>images</folder>

  <filename>N1.jpeg</filename>

  <path>/Users/asik/Desktop/ANPR/imagesN1.jpeg</path>

  <source>

    <database>Unknown</database>

  </source>

  <size>

    <width>1920</width>

    <height>1080</height>

    <depth>3</depth>

  </size>

  <segmented>0</segmented>

  <object>

    <name>number_plate</name>

    <pose>Unspecified</pose>
```

```
<truncated>0</truncated>

<difficult>0</difficult>

<bndbox>

  <xmin>1093</xmin>

  <ymin>645</ymin>

  <xmax>1396</xmax>

  <ymax>727</ymax>

</bndbox>

</object>

</annotation>
```

We need to perform some data preprocessing after you have finished the tagging process. Since the label produces XML, we need data in array format in order to use it for training. To do that, we will employ the label's useful information, which is the diagonal points of the rectangular box or bounding box, which are, respectively, xmin, ymin, xmax, and ymax. You can get this in XML format. Therefore, we must extract the data and save it in whatever format that is practical. In this case, I will convert the bounding data into CSV, which I will then use Pandas to transform into an array. Let's now examine how to use Python to parse the data.

2.4 PARSING DATA FROM XML AND CONVERTING IT INTO CSV

Firstly let me load all libraries which I will use in this project at one time. As well I will using **xml.etree** python library to parse the data from XML and also import pandas and glob. Using glob let first get all the XML files that are produced during labelling.

In [1]:

```
import os
import cv2
import numpy as np
import pandas as pd
import tensorflow as tf
import pytesseract as pt
import plotly.express as px
import matplotlib.pyplot as plt
import xml.etree.ElementTree as xet

from glob import glob
from skimage import io
from shutil import copy
from tensorflow.keras.models import Model
from tensorflow.keras.callbacks import TensorBoard
from sklearn.model_selection import train_test_split
from tensorflow.keras.applications import InceptionResNetV2
from tensorflow.keras.layers import Dense, Dropout, Flatten, Input
from tensorflow.keras.preprocessing.image import load_img, img_to_array
```


In [2]:

```
path = glob('../input/number-plate-detection/images/*.xml')
labels_dict = dict(filepath=[],xmin=[],xmax=[],ymin=[],ymax=[])
for filename in path:

    info = xet.parse(filename)
    root = info.getroot()
    member_object = root.find('object')
    labels_info = member_object.find('bndbox')
    xmin = int(labels_info.find('xmin').text)
    xmax = int(labels_info.find('xmax').text)
    ymin = int(labels_info.find('ymin').text)
    ymax = int(labels_info.find('ymax').text)

    labels_dict['filepath'].append(filename)
    labels_dict['xmin'].append(xmin)
    labels_dict['xmax'].append(xmax)
    labels_dict['ymin'].append(ymin)
    labels_dict['ymax'].append(ymax)
```

In the above code, we individually take each file and parse into xml.etree and find the object -> bndbox. Then we extract xmin,xmax,ymin,ymax and saved those values in the dictionary. After we convert it into a pandas data frame and save that into CSV file and save it in project folder as shown below.

In [3]:

```
df = pd.DataFrame(labels_dict)
df.to_csv('labels.csv',index=False)
df.head()
```

Out [3]:

	Filepath	xmin	xmax	ymin	ymax
0	../input/number-plate-detection/images/N148.xml	244	369	240	293
1	../input/number-plate-detection/images/N177.xml	331	538	263	317
2	../input/number-plate-detection/images/N173.xml	80	335	150	243
3	../input/number-plate-detection/images/N213.xml	131	209	129	153
4	../input/number-plate-detection/images/N119.xml	180	559	216	314

With the above code, we successfully extract the diagonal position of each image and convert the data from an unstructured to a structured format. You can have A look data above. Now also extract the respective image filename of the XML

In [4]:

```
filename = df['filepath'][0]
def getFilename(filename):
    filename_image = xet.parse(filename).getroot().find('filename').text
    filepath_image = os.path.join('../input/number-plate-detection/images',filename_image)
    return filepath_image
getFilename(filename)
```

out[4]:

'../input/number-plate-detection/images/N148.jpeg'

In [5]:

```
image_path = list(df['filepath'].apply(getFilename))
image_path[:10]#random check
```

Out[5]:

```
['../input/number-plate-detection/images/N148.jpeg',
 '../input/number-plate-detection/images/N177.jpeg',
 '../input/number-plate-detection/images/N173.jpeg',
 '../input/number-plate-detection/images/N213.jpeg',
 '../input/number-plate-detection/images/N119.jpeg',
 '../input/number-plate-detection/images/N103.jpeg',
 '../input/number-plate-detection/images/N166.jpeg',
 '../input/number-plate-detection/images/N127.jpeg',
 '../input/number-plate-detection/images/N162.jpeg',
 '../input/number-plate-detection/images/N198.jpeg']
```

2.5 VERIFY THE DATA

As till now we did the manual process it is important to verify the information is we got is valid or not. For that just verify the bounding box is appearing properly for a given image. Here I consider the image N2.jpeg and the corresponding diagonal position can found in df. Result you can see on *Figure 2.4*

In[6]:

```
file_path = image_path[87] #path of our image N2.jpeg
img = cv2.imread(file_path) #read the image
# xmin-1804/ymin-1734/xmax-2493/ymax-1882
img = io.imread(file_path) #Read the image
fig = px.imshow(img)
fig.update_layout(width=600, height=500, margin=dict(l=10, r=10, b=10, t=10),xaxis_title='Figure 8 - N2.jpeg with boundin g box')
fig.add_shape(type='rect',x0=1804, x1=2493, y0=1734, y1=1882, xref='x', yref='y',line_color='cyan')
```



Figure 2.4 – N2.jpeg with bounding box

3. DATA PROCESSING

3.1 READ DATA

This is a very important step, in this process we will take each and every image and convert it into an array using OpenCV and resize the image into 224 x 224 which is the standard compatible size of the pre-trained transfer learning model.

In [7]:

```
#Targeting all our values in array selecting all columns
labels = df.iloc[:,1:].values
data = []
output = []
for ind in range(len(image_path)):
    image = image_path[ind]
    img_arr = cv2.imread(image)
    h,w,d = img_arr.shape
    # Preprocessing
    load_image = load_img(image,target_size=(224,224))
    load_image_arr = img_to_array(load_image)
    norm_load_image_arr = load_image_arr/255.0 # Normalization
    # Normalization to labels
    xmin,xmax,ymin,ymax = labels[ind]
    nxmin,nxmax = xmin/w,xmax/w
    nymin,nymax = ymin/h,ymax/h
```



```
label_norm = (nxmin,nxmax,nymin,nymax) # Normalized output
# Append
data.append(norm_load_image_arr)
output.append(label_norm)
```

Next, since we know that the maximum number for an 8-bit image is $2^8 - 1 = 255$, we will simply divide the image by the maximum number to normalize it. We shall divide our image 255.0 for that reason.

Normalization is the process of analyzing an array using its maximum value (Min-Max Scaler). It's also necessary for us to normalize our labels. Since the output range for the deep learning model should fall between 0 and 1. We must divide the diagonal points by the image's width and height in order to normalize the labels. values in a Python list, lastly.

3.2 SPLIT TRAIN AND TEST SET

In the next step, we will convert the list into an array using **Numpy**.

```
In [8]:
# Convert data to array
X = np.array(data,dtype=np.float32)
y = np.array(output,dtype=np.float32)
```

Now split the data into training and testing set using **sklearn**.

```
In [9]:
# Split the data into training and testing set using sklearn.
x_train,x_test,y_train,y_test = train_test_split(X,y,train_size=0.8,random_state=0)
x_train.shape,x_test.shape,y_train.shape,y_test.shape
```

```
Out[9]:
((180, 224, 224, 3), (45, 224, 224, 3), (180, 4), (45, 4))
```

4. DEEP LEARNING FOR OBJECT DETECTION

4.1 MODEL BUILDING

A convolutional neural network called Inception-ResNet-v2 was trained using over a million pictures from the ImageNet collection. With 164 layers, the network is capable of classifying photos into 1000 different object categories, including several animals and keyboards, mice, and pencils. Consequently, a vast array of image rich feature representations have been trained by the network. The classification task was performed using the Inception-ResNet-v2. Figure 9 displays the network's architecture. The Inception structure and the Residual connection are combined to create the basis of Inception-Resnet-v2. Residual connections are used in the Inception-Resnet block to merge convolutional filters of different sizes. Residual connections are employed to shorten training times while also avoiding the degradation issue brought on by deep structures.

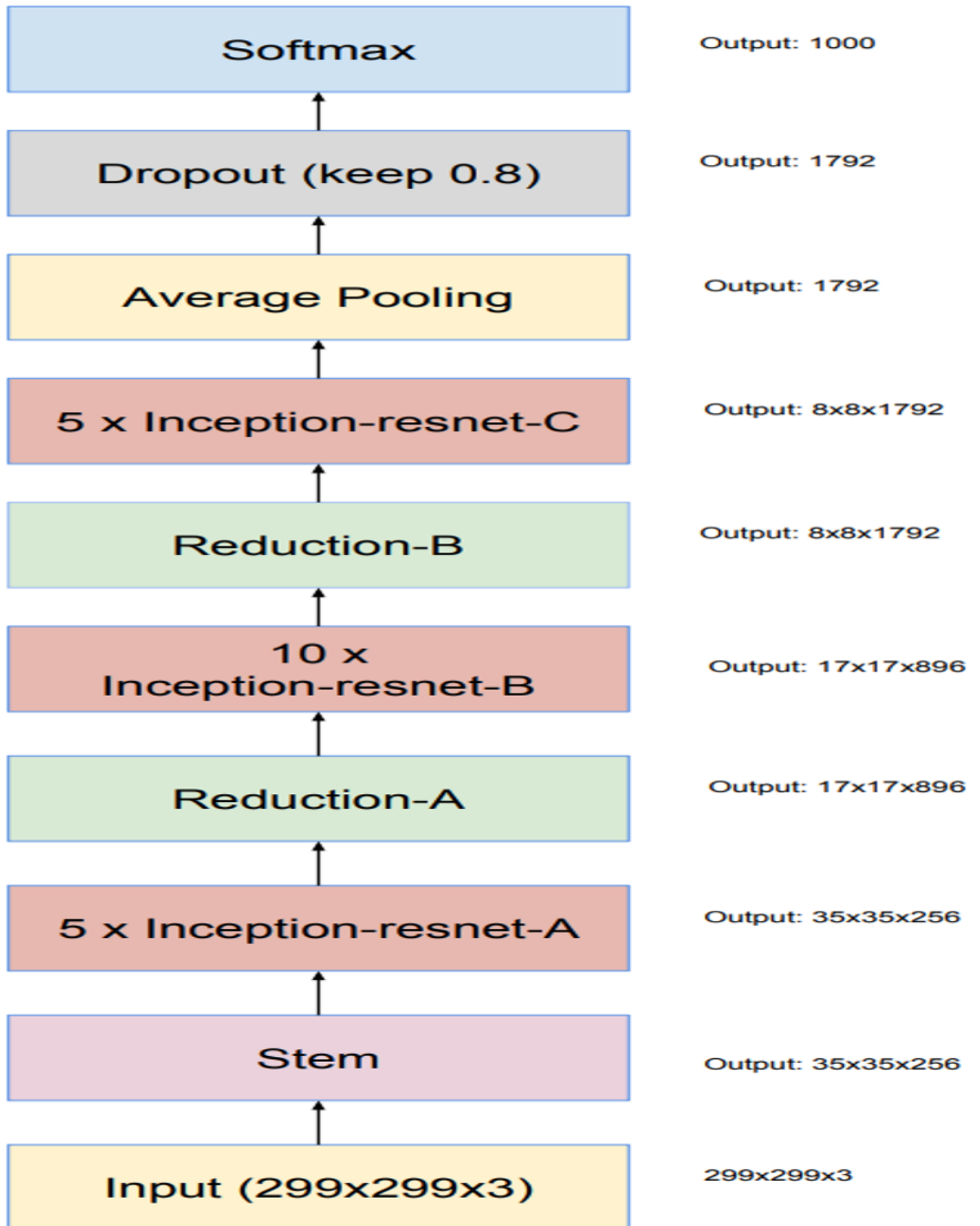


Figure 4.1 - Diagram

We are ready to train a deep learning model for object detection. Here we will use the Inception-ResNet-v2 model with pre-trained weights and train this to our data. We already import necessary libraries from TensorFlow previously, let's continue.

In [10]:

```
inception_resnet = InceptionResNetV2(weights="imagenet",include_top=False, input_tensor=Input(shape=(224,224,3)))
# -----
headmodel = inception_resnet.output
headmodel = Flatten()(headmodel)
headmodel = Dense(500,activation="relu")(headmodel)
headmodel = Dense(250,activation="relu")(headmodel)
headmodel = Dense(4,activation='sigmoid')(headmodel)

# ----- model
model = Model(inputs=inception_resnet.input,outputs=headmodel)
2022-06-17 11:06:59.087750: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2022-06-17 11:06:59.092729: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2022-06-17 11:06:59.093468: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2022-06-17 11:06:59.094728: I tensorflow/core/platform/cpu_feature_guard.cc:142] This TensorFlow binary is optimized with oneAPI Deep Neural Network Library (oneDNN) to use the following CPU instructions in performance-critical operations: AVX2 AVX512F FMA
To enable them in other operations, rebuild TensorFlow with the appropriate compiler flags.
2022-06-17 11:06:59.095013: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2022-06-17 11:06:59.095684: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2022-06-17 11:06:59.096340: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2022-06-17 11:07:01.372375: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2022-06-17 11:07:01.373241: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2022-06-17 11:07:01.373896: I tensorflow/stream_executor/cuda/cuda_gpu_executor.cc:937] successful NUMA node read from SysFS had negative value (-1), but there must be at least one NUMA node, so returning NUMA node zero
2022-06-17 11:07:01.374503: I tensorflow/core/common_runtime/gpu/gpu_device.cc:1510] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 15385 MB memory: -> device: 0, name: Tesla P100-PCIE-16GB, pci bus id: 0000:00:04.0, compute capability: 6.0
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/inception_resnet_v2/inception_resnet_v2_weights_tf_dim_ordering_tf_kernels_notop.h5
219062272/219055592 [=====] - 1s 0us/step
219070464/219055592 [=====] - 1s 0us/step
```

Now compile the model and have a look at our summary. Don't be surprised, the summary will be a bit massive. The summary is textual and includes information about: The layers and their order in the model. The output shape of each layer. The number of parameters (weights) in each layer.

In [11]:

```
# Compile model
model.compile(loss='mse',optimizer=tf.keras.optimizers.Adam(learning_rate=1e-4))
model.summary()

Model: "model"
```

Layer (type)	Output Shape	Param #	Connected to
=====			
input_1 (InputLayer)	[(None, 224, 224, 3)]	0	
conv2d (Conv2D)	(None, 111, 111, 32)	864	input_1[0][0]
batch_normalization (BatchNormaliza	(None, 111, 111, 32)	96	conv2d[0][0]
activation (Activation)	(None, 111, 111, 32)	0	batch_normalization[0][0]
conv2d_1 (Conv2D)	(None, 109, 109, 32)	9216	activation[0][0]
batch_normalization_1 (BatchNor	(None, 109, 109, 32)	96	conv2d_1[0][0]
activation_1 (Activation)	(None, 109, 109, 32)	0	batch_normalization_1[0][0]
conv2d_2 (Conv2D)	(None, 109, 109, 64)	18432	activation_1[0][0]
batch_normalization_2 (BatchNor	(None, 109, 109, 64)	192	conv2d_2[0][0]
activation_2 (Activation)	(None, 109, 109, 64)	0	batch_normalization_2[0][0]
max_pooling2d (MaxPooling2D)	(None, 54, 54, 64)	0	activation_2[0][0]
conv2d_3 (Conv2D)	(None, 54, 54, 80)	5120	max_pooling2d[0][0]
batch_normalization_3 (BatchNor	(None, 54, 54, 80)	240	conv2d_3[0][0]
activation_3 (Activation)	(None, 54, 54, 80)	0	batch_normalization_3[0][0]
conv2d_4 (Conv2D)	(None, 52, 52, 192)	138240	activation_3[0][0]
batch_normalization_4 (BatchNor	(None, 52, 52, 192)	576	conv2d_4[0][0]
activation_4 (Activation)	(None, 52, 52, 192)	0	batch_normalization_4[0][0]
max_pooling2d_1 (MaxPooling2D)	(None, 25, 25, 192)	0	activation_4[0][0]
conv2d_8 (Conv2D)	(None, 25, 25, 64)	12288	max_pooling2d_1[0][0]
batch_normalization_8 (BatchNor	(None, 25, 25, 64)	192	conv2d_8[0][0]
activation_8 (Activation)	(None, 25, 25, 64)	0	batch_normalization_8[0][0]
conv2d_6 (Conv2D)	(None, 25, 25, 48)	9216	max_pooling2d_1[0][0]
conv2d_9 (Conv2D)	(None, 25, 25, 96)	55296	activation_8[0][0]
batch_normalization_6 (BatchNor	(None, 25, 25, 48)	144	conv2d_6[0][0]
batch_normalization_9 (BatchNor	(None, 25, 25, 96)	288	conv2d_9[0][0]
activation_6 (Activation)	(None, 25, 25, 48)	0	batch_normalization_6[0][0]
activation_9 (Activation)	(None, 25, 25, 96)	0	batch_normalization_9[0][0]

-
-
-
-

batch_normalization_197 (BatchN (None, 5, 5, 224)	672	conv2d_197[0][0]
activation_197 (Activation) (None, 5, 5, 224)	0	batch_normalization_197[0][0]
conv2d_195 (Conv2D) (None, 5, 5, 192)	399360	block8_8_ac[0][0]
conv2d_198 (Conv2D) (None, 5, 5, 256)	172032	activation_197[0][0]
batch_normalization_195 (BatchN (None, 5, 5, 192)	576	conv2d_195[0][0]
batch_normalization_198 (BatchN (None, 5, 5, 256)	768	conv2d_198[0][0]
activation_195 (Activation) (None, 5, 5, 192)	0	batch_normalization_195[0][0]
activation_198 (Activation) (None, 5, 5, 256)	0	batch_normalization_198[0][0]
block8_9_mixed (Concatenate) (None, 5, 5, 448)	0	activation_195[0][0] activation_198[0][0]
block8_9_conv (Conv2D) (None, 5, 5, 2080)	933920	block8_9_mixed[0][0]
block8_9 (Lambda) (None, 5, 5, 2080)	0	block8_8_ac[0][0] block8_9_conv[0][0]
block8_9_ac (Activation) (None, 5, 5, 2080)	0	block8_9[0][0]
conv2d_200 (Conv2D) (None, 5, 5, 192)	399360	block8_9_ac[0][0]
batch_normalization_200 (BatchN (None, 5, 5, 192)	576	conv2d_200[0][0]
=====		
Total params: 73,663,490		
Trainable params: 73,602,946		
Non-trainable params: 60,544		

4.2 TRAINING AND SAVE

In [12]:

```
tfb = TensorBoard('object_detection')
history = model.fit(x=x_train,y=y_train,batch_size=10,epochs=180,
                    validation_data=(x_test,y_test),callbacks=[tfb])
```

```
18/18 [=====] - 30s 462ms/step - loss: 0.0632 - val_loss: 0.0493
Epoch 2/180
18/18 [=====] - 3s 181ms/step - loss: 0.0241 - val_loss: 0.0181
Epoch 3/180
18/18 [=====] - 3s 180ms/step - loss: 0.0089 - val_loss: 0.0171
Epoch 4/180
18/18 [=====] - 3s 177ms/step - loss: 0.0056 - val_loss: 0.0168
Epoch 5/180
18/18 [=====] - 3s 177ms/step - loss: 0.0041 - val_loss: 0.0180
Epoch 6/180
18/18 [=====] - 3s 185ms/step - loss: 0.0034 - val_loss: 0.0109
Epoch 7/180
18/18 [=====] - 3s 184ms/step - loss: 0.0029 - val_loss: 0.0113
Epoch 8/180
18/18 [=====] - 3s 180ms/step - loss: 0.0073 - val_loss: 0.0090
Epoch 9/180
18/18 [=====] - 3s 183ms/step - loss: 0.0044 - val_loss: 0.0097
Epoch 10/180
18/18 [=====] - 3s 180ms/step - loss: 0.0023 - val_loss: 0.0084
Epoch 11/180
18/18 [=====] - 3s 181ms/step - loss: 0.0018 - val_loss: 0.0094
Epoch 12/180
18/18 [=====] - 3s 180ms/step - loss: 0.0016 - val_loss: 0.0133
Epoch 13/180
18/18 [=====] - 3s 181ms/step - loss: 0.0012 - val_loss: 0.0141
Epoch 14/180
18/18 [=====] - 3s 177ms/step - loss: 0.0011 - val_loss: 0.0121
Epoch 15/180
18/18 [=====] - 3s 176ms/step - loss: 8.5752e-04 - val_loss: 0.0129
.
.
.
.
Epoch 165/180
18/18 [=====] - 3s 178ms/step - loss: 7.3352e-05 - val_loss: 0.0083
Epoch 166/180
18/18 [=====] - 3s 177ms/step - loss: 7.7341e-05 - val_loss: 0.0069
Epoch 167/180
18/18 [=====] - 3s 181ms/step - loss: 7.4907e-05 - val_loss: 0.0070
Epoch 168/180
18/18 [=====] - 4s 202ms/step - loss: 6.1625e-05 - val_loss: 0.0073
Epoch 169/180
18/18 [=====] - 3s 177ms/step - loss: 8.5513e-05 - val_loss: 0.0069
Epoch 170/180
18/18 [=====] - 3s 180ms/step - loss: 7.8864e-05 - val_loss: 0.0070
Epoch 171/180
18/18 [=====] - 3s 180ms/step - loss: 7.7951e-05 - val_loss: 0.0070
```

```
Epoch 172/180
18/18 [=====] - 3s 177ms/step - loss: 7.2048e-05 - val_loss: 0.0072
Epoch 173/180
18/18 [=====] - 3s 178ms/step - loss: 5.7776e-05 - val_loss: 0.0068
Epoch 174/180
18/18 [=====] - 3s 181ms/step - loss: 8.3889e-05 - val_loss: 0.0078
Epoch 175/180
18/18 [=====] - 3s 181ms/step - loss: 8.6761e-05 - val_loss: 0.0089
Epoch 176/180
18/18 [=====] - 3s 177ms/step - loss: 9.1527e-05 - val_loss: 0.0080
Epoch 177/180
18/18 [=====] - 3s 178ms/step - loss: 6.3443e-05 - val_loss: 0.0066
Epoch 178/180
18/18 [=====] - 4s 202ms/step - loss: 6.3016e-05 - val_loss: 0.0064
Epoch 179/180
18/18 [=====] - 3s 181ms/step - loss: 6.6962e-05 - val_loss: 0.0068
Epoch 180/180
18/18 [=====] - 3s 180ms/step - loss: 6.2468e-05 - val_loss: 0.0066
```

```
In [13]: model.save('./object_detection.h5')
```

5. PIPELINE OBJECT DETECTION MODEL

5.1 MAKE PREDICTIONS

This is the final step in object detection. In this step, we will put it all together and get the prediction for a given image. First, I would like to try with one of my test pictures of car. Let load our model.

```
In [14]:
# Load model
model = tf.keras.models.load_model('./object_detection.h5')
print('Model loaded Sucessfully')
```

Model loaded Successfully

Next is loading our TEST picture with right path to it. I loaded some more images for this purpose only - folder **TEST**.

In [15]:

```
path = '../input/number-plate-detection/TEST/TEST.jpeg'
image = load_img(path) # PIL object
image = np.array(image,dtype=np.uint8) # 8 bit array (0,255)
image1 = load_img(path,target_size=(224,224))
image_arr_224 = img_to_array(image1)/255.0 # Convert into array and get the normalized output

# Size of the orginal image
h,w,d = image.shape
print('Height of the image =',h)
print('Width of the image =',w)
```

Height of the image = 729

Width of the image = 901

Now we can have a look at our image Figure 5.1


```
In[16]:  
fig = px.imshow(image)  
fig.update_layout(width=700, height=500, margin=dict(l=10, r=10, b=10, t=10), xaxis_title='Figure 13 - TEST Image')
```



Figure 5.1 – Test Image

So, let's look into the shape of my image

```
In[17]: image_arr_224.shape
```

```
Out[17]: (224, 224, 3)
```

But in order to pass this image of a model, we need to provide the data in the dynamic fourth dimension. And what one indicates is a number of images. So here we are just passing only one image.

```
In[18]:
```

```
test_arr = image_arr_224.reshape(1,224,224,3)  
test_arr.shape
```

```
Out[18]: (1, 224, 224, 3)
```

5.2 DE-NORMALIZE THE OUTPUT

In[19]:

```
# Make predictions
coords = model.predict(test_arr)
coords
```

Out[19]:

```
array([[0.35251912, 0.63733816, 0.65963215, 0.7349673 ]], dtype=float32)
```

The output that we obtained from the model is the output that has been normalized. Therefore, we must convert back to our original form values, which we actually accomplished throughout the training phase. throughout that process, we had the original form values and converted them to their normalized form. We will essentially de-normalize the values once more.

In[20]:

```
# Denormalize the values
denorm = np.array([w,w,h,h])
coords = coords * denorm
coords
```

Out[20]:

```
array([[317.6197314 , 574.24168348, 480.8718347 , 535.7911554 ]])
```

5.3 BOUNDING BOX

Now we will draw bounding box on top of the image. I just want to provide the two diagonal points. Let's make use of these points and let's draw the rectangle box.

In[21]:

```
coords = coords.astype(np.int32)
coords
```

Out[21]:

```
array([[317, 574, 480, 535]], dtype=int32)
```

In[22]:

```
# Draw bounding on top the image
xmin, xmax,ymin,ymax = coords[0]
pt1 =(xmin,ymin)
pt2 =(xmax,ymax)
print(pt1, pt2)
```

```
(317, 480) (574, 535)
```

In[23]:

```
cv2.rectangle(image,pt1,pt2,(0,255,0),3)
fig = px.imshow(image)
fig.update_layout(width=700, height=500, margin=dict(l=10, r=10, b=10, t=10))
```



Figure 5.2 – Detecting Number Plate

5.4 CREATE PIPELINE

Now what we'll do, let's put it all together in one place and create function. And in the end visualize it. Our output will return image and coordinates of bounding box.

In[24]:

```
# Create pipeline
path = '../input/number-plate-detection/TEST/TEST.jpeg'
def object_detection(path):

    # Read image
    image = load_img(path) # PIL object
    image = np.array(image,dtype=np.uint8) # 8 bit array (0,255)
    image1 = load_img(path,target_size=(224,224))

    # Data preprocessing
    image_arr_224 = img_to_array(image1)/255.0 # Convert to array & normalized
    h,w,d = image.shape
    test_arr = image_arr_224.reshape(1,224,224,3)
```

```

# Make predictions
coords = model.predict(test_arr)

# Denormalize the values
denorm = np.array([w,w,h,h])
coords = coords * denorm
coords = coords.astype(np.int32)

# Draw bounding on top the image
xmin, xmax,ymin,ymax = coords[0]
pt1 =(xmin,ymin)
pt2 =(xmax,ymax)
print(pt1, pt2)
cv2.rectangle(image,pt1,pt2,(0,255,0),3)
return image, coords

image, cods = object_detection(path)

fig = px.imshow(image)
fig.update_layout(width=700, height=500, margin=dict(l=10, r=10, b=10, t=10),xaxis_title='Figure 14')

```

(317, 480) (574, 535)



Figure 5.3 – Detecting Number Plate after Pipeline

6. OPTICAL CHARACTER RECOGNITION - OCR

6.1 TESSERACT OCR

Optical character recognition (OCR) software that is used to extract text from the image. Tesseract OCR have a python API and it is open source. Firstly, we will do installation of it. It pretty simple and depend on your OS.

6.2 LIMITATIONS OF PYTESSERACT

When the text in the foreground is clearly separated from the background, Tesseract performs at its best. It might be quite difficult to guarantee these kinds of configurations in practice. There are several reasons why Tesseract may not produce high-quality results, such as background noise in the image. The recognition result improves with increased image quality (size, contrast, lightning). To achieve better OCR results, some preprocessing is necessary. Text must be oriented horizontally, photos must have the maximum amount of contrast, and scaling must be done correctly. Despite its great power, Tesseract OCR has the following drawbacks.

Tesseract limitations summed in the list

- The OCR is not as precise as a few of the commercial options we have.
- performs poorly on photos that have artifacts such as complicated backgrounds, warped perspective, and partial occlusion.
- It cannot identify handwritten letters.
- It might identify nonsense and record it as OCR output.
- Results can be subpar if a document comprises languages other than those listed via the -l LANG parameters.
- Its ability to analyze documents' natural reading order isn't always reliable. For instance, it might attempt to combine text across columns if it is unable to detect that a document has two columns.
- Low-quality scans can result in low-quality OCR.
- It withholds information regarding the font family to which the text belongs.

6.3 EXTRACT NUMBER PLATE TEXT FROM IMAGE

Firstly, we will load our image and convert to array. Crop our bounding box with coordinates of it. We will identify region of interest (ROI) and have look at our cropped image *Figure 6.1*.

In[25]:

```
img = np.array(load_img(path))
xmin ,xmax,ymin,ymax = cods[0]
roi = img[ymin:ymax,xmin:xmax]
fig = px.imshow(roi)
fig.update_layout(width=350, height=250, margin=dict(l=10, r=10, b=10, t=10),xaxis_title='Figure 15 Cropped image')
```




Figure 6.1 – Cropped Image

With use of tesseract, we will extract the text from the image.

In[26]:

```
# extract text from image
text = pt.image_to_string(roi)
print(text)
```

FKL Si K 4999

Granted, the correct text was not sent, but at least 90% of the information is still accessible. It's only one example, and once more, it's important to note that more data will lead to better predictions. We will eventually reach that stage. What I've discovered here: To start, we don't have a lot of data to tackle this issue, therefore I added more sets of data—virtually identical sets from other kagglers who just posted—to this discussion. To be honest, I don't think this model performs well. However, the entire process that was completed allowed us to understand the concept, and with YOLO's assistance, we will build another model where we can observe how it performs.

7. REAL TIME NUMBER PLATE RECOGNITION WITH YOLO

7.1 EXPLANATION OF REQUIRED DATA

One of the most popular deep learning-based object identification algorithms available is called You Only Look Once, or YOLO. An image is divided into grid systems by YOLO, and each grid recognizes items inside it. Based on the data streams, they can be utilized for real-time object detection. They don't need a lot of processing power. YOLOv5's network architecture. There are three components to it: 1) Yolo Layer as the head, 2) PANet as the neck, and 3) CSPDarknet as the backbone. Prior to being supplied to PANet for feature fusion, the data are first entered to CSPDarknet for feature extraction. The detection results (class, score, position, and size) are finally output by Yolo Layer.

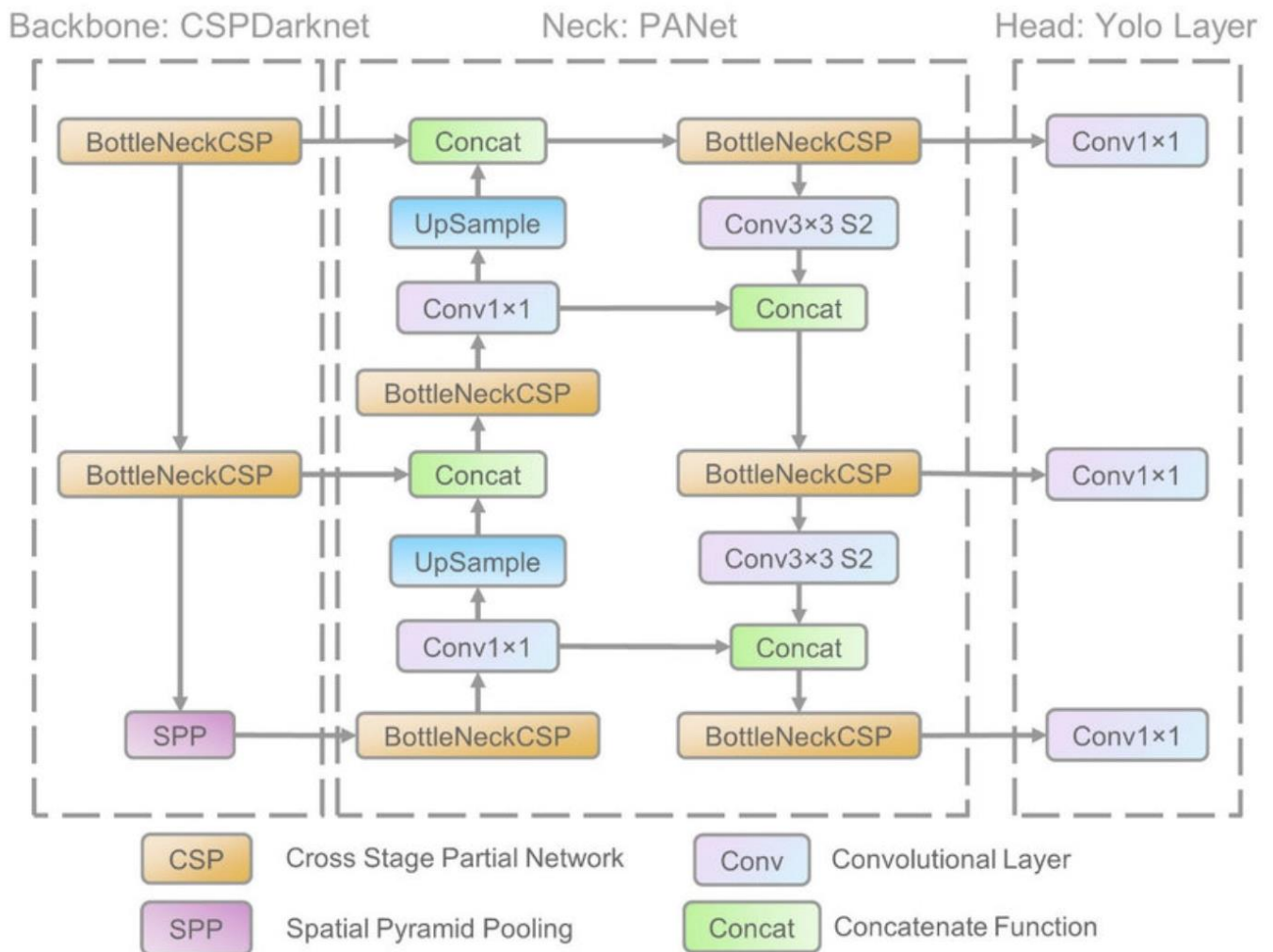


Figure 7.1 – Procedure Diagram

Although our prior procedure is excellent, the model has an issue. The main issues with the model are its extremely poor precision in identifying license plates and its extremely slow processing speed. YOLO is the most potent reduction model currently available, and it will be used to get around the aforementioned issues, especially the precision issue. Let's now examine how we can use the data preparation we now have to our YOLO model. The Yola model can be used with the labeling and data pretreatment work that we have completed for the current effort. However, there is one significant change we must make to YOLO: our label. As you previously saw, we made advantage of the xmin, xmax, ymin, ymax and our real output is VI Max. However, with YOLO, the bounding box's width, height, and center are all included. In particular, the X-Y position X refers to the bounding box's center. Allow me to illustrate with an example. Figure 7.2. The center X and Y of this image should be labeled as the YOLO, and the normalized width and height of the image should be used as the label. Furthermore, W and H represent the bounding box's width and height, respectively, normalized to the image's width and height.

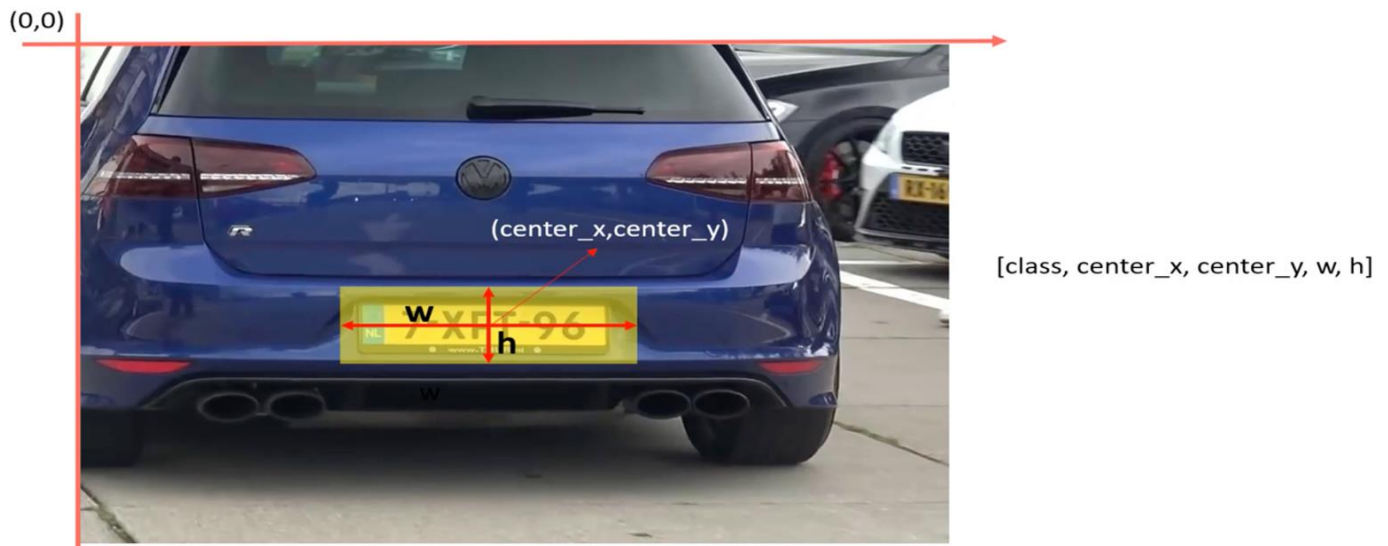


Figure 7.2 – Measuring width and height of number plate

In other words, this is the format we now need to prepare our data in for the labeling process, along with the center location of X and the center position of Y. this is referring to the bounding box's height, breadth, and dimensions. [class, w, h, center_x, center_y]. That's one of the adjustments we must make. The next item is that the folder structure we have to set up will resemble Figure 7.3, especially for the YOLO.

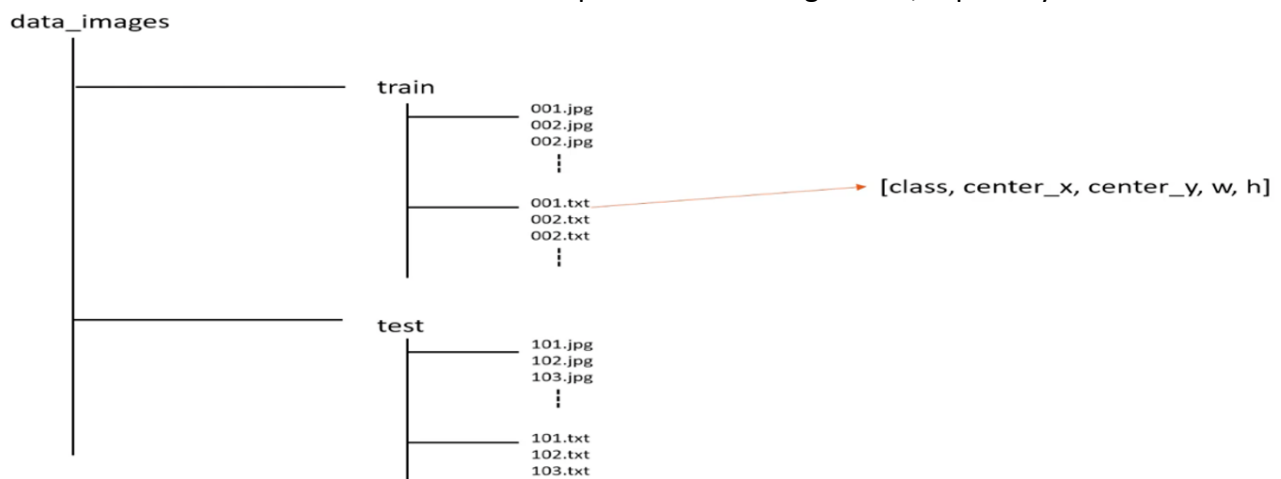


Figure 7.3 – Diagram of process of training and testing

Now let's look at the data images. We should train and test using the data photos. All of the training-related photos must be kept in the training test folder, together with the associated labels in.txt format. Furthermore, the label data should have the same name as our image. We will prepare our test data in the same manner as our training data.

I will create a function and attempt to extract the file name, weight, and height from the XML files using that function. constructing a function that will parse and return our filename, weight, and height; apply all of this to our data frame as seen below.

In[27]:

```
# parsing
def parsing(path):
    parser = xet.parse(path).getroot()
    name = parser.find('filename').text
    filename = f'../input/number-plate-detection/images/{name}'
```



```
# width and height
parser_size = parser.find('size')
width = int(parser_size.find('width').text)
height = int(parser_size.find('height').text)

return filename, width, height
df[['filename','width','height']] = df['filepath'].apply(parsing).apply(pd.Series)
df.head()
```

Out[27]:

	filepath	xmin	xmax	ymin	ymax	filename	width	height
0	../input/number-plate-detection/images/N148.xml	244	369	240	293	../input/number-plate-detection/images/N148.jpeg	600	430
1	../input/number-plate-detection/images/N177.xml	331	538	263	317	../input/number-plate-detection/images/N177.jpeg	850	681
2	../input/number-plate-detection/images/N173.xml	80	335	150	243	../input/number-plate-detection/images/N173.jpeg	445	349
3	../input/number-plate-detection/images/N213.xml	131	209	129	153	../input/number-plate-detection/images/N213.jpeg	350	196
4	../input/number-plate-detection/images/N119.xml	180	559	216	314	../input/number-plate-detection/images/N119.jpeg	678	475

Now the next step is let me calculate the center_x, center_y, width and height, which is normalized to width and height. And as well normalize width and height of bounding box.

In[28]:

```
# center_x, center_y, width , height
df['center_x'] = (df['xmax'] + df['xmin'])/(2*df['width'])
df['center_y'] = (df['ymax'] + df['ymin'])/(2*df['height'])

df['bb_width'] = (df['xmax'] - df['xmin'])/df['width']
df['bb_height'] = (df['ymax'] - df['ymin'])/df['height']
df.head()
```

Out[28]:

	filepath	xmin	xmax	ymin	ymax	filename	width	height	center_x	center_y	bb_width	bb_height
0	../input/number-plate-detection/images/N148.xml	244	369	240	293	../input/number-plate-detection/images/N148.jpeg	600	430	0.510833	0.619767	0.208333	0.123256
1	../input/number-plate-detection/images/N177.xml	331	538	263	317	../input/number-plate-detection/images/N177.jpeg	850	681	0.511176	0.425844	0.243529	0.079295
2	../input/number-plate-detection/images/N173.xml	80	335	150	243	../input/number-plate-detection/images/N173.jpeg	445	349	0.466292	0.563037	0.573034	0.266476
3	../input/number-plate-detection/images/N213.xml	131	209	129	153	../input/number-plate-detection/images/N213.jpeg	350	196	0.485714	0.719388	0.222857	0.122449
4	../input/number-plate-detection/images/N119.xml	180	559	216	314	../input/number-plate-detection/images/N119.jpeg	678	475	0.544985	0.557895	0.558997	0.206316

7.2 DATA PREPARATION

One of the major challenges in the training of YOLO is hardware itself. YOLO requires a fast-processing chip used to get trained. It is extremely difficult to train the YOLO in normal CPUs. Firstly, we have to clone YOLOV5 to our work space. Make sure you in right folder type as below, let's do it.

```
In[29]: !git clone https://github.com/ultralytics/yolov5
```

```
Cloning into 'yolov5'...
remote: Enumerating objects: 12264, done.
remote: Counting objects: 100% (35/35), done.
remote: Compressing objects: 100% (22/22), done.
remote: Total 12264 (delta 18), reused 22 (delta 13), pack-reused 12229
Receiving objects: 100% (12264/12264), 11.95 MiB | 21.74 MiB/s, done.
Resolving deltas: 100% (8488/8488), done.
```

```
In[30]: !pip install -r ./yolov5/requirements.txt
```

Requirement already satisfied: matplotlib>=3.2.2 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 5)) (3.5.2)

Requirement already satisfied: numpy>=1.18.5 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 6)) (1.21.6)

Requirement already satisfied: opencv-python>=4.1.1 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 7)) (4.5.4.60)

Requirement already satisfied: Pillow>=7.1.2 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 8)) (9.1.0)

Requirement already satisfied: PyYAML>=5.3.1 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 9)) (6.0)

Requirement already satisfied: requests>=2.23.0 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 10)) (2.27.1)

Requirement already satisfied: scipy>=1.4.1 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 11)) (1.7.3)

Requirement already satisfied: torch>=1.7.0 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 12)) (1.11.0)

Requirement already satisfied: torchvision>=0.8.1 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 13)) (0.12.0)

Requirement already satisfied: tqdm>=4.41.0 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 14)) (4.64.0)

Requirement already satisfied: protobuf<=3.20.1 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 15)) (3.20.1)

Requirement already satisfied: tensorboard>=2.4.1 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 18)) (2.6.0)

Requirement already satisfied: pandas>=1.1.4 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 22)) (1.3.5)

Requirement already satisfied: seaborn>=0.11.0 in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 23)) (0.11.2)

Requirement already satisfied: ipython in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 35)) (7.33.0)

Requirement already satisfied: psutil in /opt/conda/lib/python3.7/site-packages (from -r ./yolov5/requirements.txt (line 36)) (5.9.0)

Collecting thop

Downloading thop-0.1.0.post2206102148-py3-none-any.whl (15 kB)

Requirement already satisfied: packaging>=20.0 in /opt/conda/lib/python3.7/site-packages (from matplotlib>=3.2.2->-r ./yolov5/requirements.txt (line 5)) (21.3)

Requirement already satisfied: python-dateutil>=2.7 in /opt/conda/lib/python3.7/site-packages (from matplotlib>=3.2.2->-r ./yolov5/requirements.txt (line 5)) (2.8.2)

Requirement already satisfied: fonttools>=4.22.0 in /opt/conda/lib/python3.7/site-packages (from matplotlib>=3.2.2->-r ./yolov5/requirements.txt (line 5)) (4.33.3)

Requirement already satisfied: kiwisolver>=1.0.1 in /opt/conda/lib/python3.7/site-packages (from matplotlib>=3.2.2->-r ./yolov5/requirements.txt (line 5)) (1.4.2)

Requirement already satisfied: cycler>=0.10 in /opt/conda/lib/python3.7/site-packages (from matplotlib>=3.2.2->-r ./yolov5/requirements.txt (line 5)) (0.11.0)

Requirement already satisfied: pyparsing>=2.2.1 in /opt/conda/lib/python3.7/site-packages (from matplotlib>=3.2.2->-r ./yolov5/requirements.txt (line 5)) (3.0.9)

Requirement already satisfied: charset-normalizer~2.0.0 in /opt/conda/lib/python3.7/site-packages (from requests>=2.23.0->-r ./yolov5/requirements.txt (line 10)) (2.0.12)

Requirement already satisfied: idna<4,>=2.5 in /opt/conda/lib/python3.7/site-packages (from requests>=2.23.0->-r ./yolov5/requirements.txt (line 10)) (3.3)

Requirement already satisfied: urllib3<1.27,>=1.21.1 in /opt/conda/lib/python3.7/site-packages (from requests>=2.23.0->-r ./yolov5/requirements.txt (line 10)) (1.26.9)

Requirement already satisfied: certifi>=2017.4.17 in /opt/conda/lib/python3.7/site-packages (from requests>=2.23.0->-r ./yolov5/requirements.txt (line 10)) (2022.5.18.1)

Requirement already satisfied: typing-extensions in /opt/conda/lib/python3.7/site-packages (from torch>=1.7.0->-r ./yolov5/requirements.txt (line 12)) (4.2.0)

Requirement already satisfied: google-auth-oauthlib<0.5,>=0.4.1 in /opt/conda/lib/python3.7/site-packages (from tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (0.4.6)

Requirement already satisfied: tensorboard-plugin-wit>=1.6.0 in /opt/conda/lib/python3.7/site-packages (from tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (1.8.1)

Requirement already satisfied: werkzeug>=0.11.15 in /opt/conda/lib/python3.7/site-packages (from tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (2.1.2)

Requirement already satisfied: wheel>=0.26 in /opt/conda/lib/python3.7/site-packages (from tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (0.37.1)

Requirement already satisfied: setuptools>=41.0.0 in /opt/conda/lib/python3.7/site-packages (from tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (59.8.0)

Requirement already satisfied: grpcio>=1.24.3 in /opt/conda/lib/python3.7/site-packages (from tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (1.43.0)

Requirement already satisfied: google-auth<2,>=1.6.3 in /opt/conda/lib/python3.7/site-packages (from tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (1.35.0)

Requirement already satisfied: tensorboard-data-server<0.7.0,>=0.6.0 in /opt/conda/lib/python3.7/site-packages (from tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (0.6.1)

Requirement already satisfied: absl-py>=0.4 in /opt/conda/lib/python3.7/site-packages (from tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (1.0.0)

Requirement already satisfied: markdown>=2.6.8 in /opt/conda/lib/python3.7/site-packages (from tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (3.3.7)

Requirement already satisfied: pytz>=2017.3 in /opt/conda/lib/python3.7/site-packages (from pandas>=1.1.4->-r ./yolov5/requirements.txt (line 22)) (2022.1)

Requirement already satisfied: jedi>=0.16 in /opt/conda/lib/python3.7/site-packages (from ipython->-r ./yolov5/requirements.txt (line 35)) (0.18.1)

Requirement already satisfied: decorator in /opt/conda/lib/python3.7/site-packages (from ipython->-r ./yolov5/requirements.txt (line 35)) (5.1.1)

Requirement already satisfied: pygments in /opt/conda/lib/python3.7/site-packages (from ipython->-r ./yolov5/requirements.txt (line 35)) (2.12.0)

Requirement already satisfied: traitlets>=4.2 in /opt/conda/lib/python3.7/site-packages (from ipython->-r ./yolov5/requirements.txt (line 35)) (5.2.0)

Requirement already satisfied: matplotlib-inline in /opt/conda/lib/python3.7/site-packages (from ipython->-r ./yolov5/requirements.txt (line 35)) (0.1.3)

Requirement already satisfied: pickleshare in /opt/conda/lib/python3.7/site-packages (from ipython->-r ./yolov5/requirements.txt (line 35)) (0.7.5)

Requirement already satisfied: pexpect>4.3 in /opt/conda/lib/python3.7/site-packages (from ipython->-r ./yolov5/requirements.txt (line 35)) (4.8.0)

Requirement already satisfied: prompt-toolkit!=3.0.0,!<3.0.1,<3.1.0,>=2.0.0 in /opt/conda/lib/python3.7/site-packages (from ipython->-r ./yolov5/requirements.txt (line 35)) (3.0.29)

Requirement already satisfied: backcall in /opt/conda/lib/python3.7/site-packages (from ipython->-r ./yolov5/requirements.txt (line 35)) (0.2.0)

Requirement already satisfied: six in /opt/conda/lib/python3.7/site-packages (from absl-py>=0.4->tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (1.16.0)

Requirement already satisfied: rsa<5,>=3.1.4 in /opt/conda/lib/python3.7/site-packages (from google-auth<2,>=1.6.3->tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (4.8)

Requirement already satisfied: cachetools<5.0,>=2.0.0 in /opt/conda/lib/python3.7/site-packages (from google-auth<2,>=1.6.3->tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (4.2.4)

Requirement already satisfied: pyasn1-modules>=0.2.1 in /opt/conda/lib/python3.7/site-packages (from google-auth<2,>=1.6.3->tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (0.2.7)

Requirement already satisfied: requests-oauthlib>=0.7.0 in /opt/conda/lib/python3.7/site-packages (from google-auth-oauthlib<0.5,>=0.4.1->tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (1.3.1)

Requirement already satisfied: parso<0.9.0,>=0.8.0 in /opt/conda/lib/python3.7/site-packages (from jedi>=0.16->ipython->-r ./yolov5/requirements.txt (line 35)) (0.8.3)

Requirement already satisfied: importlib-metadata>=4.4 in /opt/conda/lib/python3.7/site-packages (from markdown>=2.6.8->tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (4.11.4)

Requirement already satisfied: ptyprocess>=0.5 in /opt/conda/lib/python3.7/site-packages (from pexpect>4.3->ipython->-r ./yolov5/requirements.txt (line 35)) (0.7.0)

Requirement already satisfied: wcwidth in /opt/conda/lib/python3.7/site-packages (from prompt-toolkit!=3.0.0,!3.0.1,<3.1.0,>=2.0.0->ipython->-r ./yolov5/requirements.txt (line 35)) (0.2.5)

Requirement already satisfied: zipp>=0.5 in /opt/conda/lib/python3.7/site-packages (from importlib-metadata>=4.4->markdown>=2.6.8->tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (3.8.0)

Requirement already satisfied: pyasn1<0.5.0,>=0.4.6 in /opt/conda/lib/python3.7/site-packages (from pyasn1-modules>=0.2.1->google-auth<2,>=1.6.3->tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (0.4.8)

Requirement already satisfied: oauthlib>=3.0.0 in /opt/conda/lib/python3.7/site-packages (from requests-oauthlib>=0.7.0->google-auth-oauthlib<0.5,>=0.4.1->tensorboard>=2.4.1->-r ./yolov5/requirements.txt (line 18)) (3.2.0)

Installing collected packages: thop

Successfully installed thop-0.1.0.post2206102148

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

We need to make a one more folder and make into it two more - name it as **data_images** and **train** and **test**. We will split data to train and split.

```
In[31]: mkdir /kaggle/working/yolov5/data_images/
```

```
In[32]: mkdir /kaggle/working/yolov5/data_images/test/
```

```
In[33]: mkdir /kaggle/working/yolov5/data_images/train/
```

```
In[34]: ### split the data into train and test
df_train = df.iloc[:200]
df_test = df.iloc[200:]
```

We will copy each image into the folder train and test and generate .txt which has label info.

```
In[35]: train_folder = './yolov5/data_images/train'
```

```
values = df_train[['filename','center_x','center_y','bb_width','bb_height']].values
```

for fname, x,y, w, h in values:

```
    image_name = os.path.split(fname)[-1]
```

```
    txt_name = os.path.splitext(image_name)[0]
```

```
    dst_image_path = os.path.join(train_folder,image_name)
```

```
    dst_label_file = os.path.join(train_folder,txt_name+'.txt')
```

```
    # copy each image into the folder
```

```
    copy(fname,dst_image_path)
```

```
    # generate .txt which has label info
```

```
    label_txt = f'O {x} {y} {w} {h}'
```

```
    with open(dst_label_file,mode='w') as f:
```

```
        f.write(label_txt)
```

```
    f.close()
```

```
test_folder = './yolov5/data_images/test'
```

```
values = df_test[['filename','center_x','center_y','bb_width','bb_height']].values
```

for fname, x,y, w, h in values:

```
    image_name = os.path.split(fname)[-1]
```

```
    txt_name = os.path.splitext(image_name)[0]
```

```
    dst_image_path = os.path.join(test_folder,image_name)
```

```
    dst_label_file = os.path.join(test_folder,txt_name+'.txt')
```

```
# copy each image into the folder
copy(fname,dst_image_path)

# generate .txt which has label info
label_txt = f'0 {x} {y} {w} {h}'
with open(dst_label_file,mode='w') as f:
    f.write(label_txt)

f.close()
```

As well I will create **data.yaml** file with path to train and validation, as well number of classes as 1, name it in list as license plate. You will need to save this in your dataset because we will need to provide the path on it later.

```
train: data_images/train
val: data_images/test
nc: 1
names: [
    'license_plate'
]
```

7.3 TRAINING YOLO

Next step is basically train model. It could take time be ready for it. You can use Kaggle or GoogleColab for it. We already set out data.yaml file, lets us give path to it and train model. Below you will find code to cleare you GPU

```
In[36]: !pip install GPUUtil

import torch
from GPUUtil import showUtilization as gpu_usage
from numba import cuda

def free_gpu_cache():
    print("Initial GPU Usage")
    gpu_usage()

    torch.cuda.empty_cache()

    cuda.select_device(0)
    cuda.close()
    cuda.select_device(0)

    print("GPU Usage after emptying the cache")
    gpu_usage()

free_gpu_cache()
```

Collecting GPUUtil

```
Downloading GPUUtil-1.4.0.tar.gz (5.5 kB)
Preparing metadata (setup.py) ... - done
Building wheels for collected packages: GPUUtil
Building wheel for GPUUtil (setup.py) ... - \ done
Created wheel for GPUUtil: filename=GPUUtil-1.4.0-py3-none-any.whl size=7411 sha256=cc480cf92970055e92e6f363d9212
080b9e8864225da31712be62e02e5f746b7
Stored in directory: /root/.cache/pip/wheels/6e/f8/83/534c52482d6da64622ddb72cd93c35d2ef2881b78fd08ff0c
```

30 | Page


```

11      -1 1      0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
12     [-1, 6] 1      0 models.common.Concat [1]
13      -1 1 361984 models.common.C3 [512, 256, 1, False]
14      -1 1 33024 models.common.Conv [256, 128, 1, 1]
15      -1 1      0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
16     [-1, 4] 1      0 models.common.Concat [1]
17      -1 1 90880 models.common.C3 [256, 128, 1, False]
18      -1 1 147712 models.common.Conv [128, 128, 3, 2]
19     [-1, 14] 1      0 models.common.Concat [1]
20      -1 1 296448 models.common.C3 [256, 256, 1, False]
21      -1 1 590336 models.common.Conv [256, 256, 3, 2]
22     [-1, 10] 1      0 models.common.Concat [1]
23      -1 1 1182720 models.common.C3 [512, 512, 1, False]
24 [17, 20, 23] 1 16182 models.yolo.Detect [1, [[10, 13, 16, 30, 33, 23], [30, 61, 62, 45, 59, 119], [116, 90
, 156, 198, 373, 326]], [128, 256, 512]]

```

YOLOv5s summary: 270 layers, 7022326 parameters, 7022326 gradients

Transferred 342/349 items from yolov5/yolov5s.pt

AMP: checks passed ☒

Scaled weight_decay = 0.0005

optimizer: SGD with parameter groups 57 weight (no decay), 60 weight, 60 bias

augmentations: Blur(always_apply=False, p=0.01, blur_limit=(3, 7)), MedianBlur(always_apply=False, p=0.01, blur_limit=(3, 7)), ToGray(always_apply=False, p=0.01), CLAHE(always_apply=False, p=0.01, clip_limit=(1, 4.0), tile_grid_size=(8, 8))

train: Scanning '/kaggle/working/yolov5/data_images/train' images and labels...2

train: New cache created: /kaggle/working/yolov5/data_images/train.cache

val: Scanning '/kaggle/working/yolov5/data_images/test' images and labels...25 f

val: New cache created: /kaggle/working/yolov5/data_images/test.cache

Plotting labels to yolov5/runs/train/Model/labels.jpg...

AutoAnchor: 3.46 anchors/target, 1.000 Best Possible Recall (BPR). Current anchors are a good fit to dataset ☒

Image sizes 640 train, 640 val

Using 2 dataloader workers

Logging results to **yolov5/runs/train/Model**

Starting training for 100 epochs...

```

Epoch gpu_mem  box  obj  cls  labels img_size
0/99  1.94G  0.1113  0.02923  0    9   640: 100%|██████████
      Class  Images  Labels    P    R  mAP@.5 mAP@.95
      Class  Images  Labels    P    R  mAP@.5 mAP@.95
      all    25    23  0.00626  0.0435  0.00477  0.000584

```

```

Epoch gpu_mem  box  obj  cls  labels img_size
1/99  2.02G  0.0895  0.02403  0   10   640: 100%|██████████
      Class  Images  Labels    P    R  mAP@.5 mAP@.95
      all    25    25  0.00267  0.8    0.0287  0.00577

```

```

Epoch gpu_mem  box  obj  cls  labels img_size
2/99  2.02G  0.08548  0.02064  0   15   640: 100%|██████████
      Class  Images  Labels    P    R  mAP@.5 mAP@.95
      all    25    25  0.00253  0.76  0.0304  0.00464

```

```

Epoch gpu_mem  box  obj  cls  labels img_size
3/99  2.02G  0.08297  0.01755  0   13   640: 100%|██████████
      Class  Images  Labels    P    R  mAP@.5 mAP@.95
      all    25    25  0.0028  0.84  0.0558  0.00967

```

```

Epoch gpu_mem  box  obj  cls  labels img_size

```

4/99 2.02G 0.07817 0.01615 0 16 640: 100%|
Class Images Labels P R mAP@.5 mAP@
all 25 25 0.00307 0.92 0.156 0.036

Epoch gpu_mem box obj cls labels img_size
5/99 2.02G 0.06983 0.01898 0 11 640: 100%|
Class Images Labels P R mAP@.5 mAP@
all 25 25 0.0766 0.72 0.382 0.151

Epoch gpu_mem box obj cls labels img_size
6/99 2.02G 0.06894 0.01614 0 18 640: 100%|
Class Images Labels P R mAP@.5 mAP@
all 25 25 0.544 0.717 0.641 0.314

Epoch gpu_mem box obj cls labels img_size
7/99 2.02G 0.0568 0.01708 0 15 640: 100%|
Class Images Labels P R mAP@.5 mAP@
all 25 25 0.55 0.48 0.44 0.0787

Epoch gpu_mem box obj cls labels img_size
8/99 2.02G 0.05308 0.01872 0 26 640: 100%|
Class Images Labels P R mAP@.5 mAP@
all 25 25 0.756 0.621 0.671 0.177

Epoch gpu_mem box obj cls labels img_size
9/99 2.02G 0.05659 0.01573 0 14 640: 100%|
Class Images Labels P R mAP@.5 mAP@
all 25 25 0.95 0.753 0.835 0.313

Epoch gpu_mem box obj cls labels img_size
10/99 2.02G 0.05794 0.01519 0 15 640: 100%|
Class Images Labels P R mAP@.5 mAP@
all 25 25 0.952 0.79 0.883 0.516

.
.
.
.

Epoch gpu_mem box obj cls labels img_size
90/99 2.02G 0.01719 0.00589 0 16 640: 100%|
Class Images Labels P R mAP@.5 mAP@
all 25 25 1 0.993 0.995 0.706

Epoch gpu_mem box obj cls labels img_size
91/99 2.02G 0.01795 0.00593 0 14 640: 100%|
Class Images Labels P R mAP@.5 mAP@
all 25 25 1 0.993 0.995 0.694

Epoch gpu_mem box obj cls labels img_size
92/99 2.02G 0.01761 0.005412 0 18 640: 100%|
Class Images Labels P R mAP@.5 mAP@
all 25 25 1 0.994 0.995 0.681

Epoch gpu_mem box obj cls labels img_size
93/99 2.02G 0.01683 0.004922 0 19 640: 100%|
Class Images Labels P R mAP@.5 mAP@

all	25	25	1	0.994	0.995	0.682
-----	----	----	---	-------	-------	-------

Epoch	gpu_mem	box	obj	cls	labels	img_size
94/99	2.02G	0.0169	0.00542	0	14	640: 100% ██████████
Class	Images	Labels	P	R	mAP@.5	mAP@
all	25	25	1	0.994	0.995	0.701

Epoch	gpu_mem	box	obj	cls	labels	img_size
95/99	2.02G	0.01639	0.005673	0	13	640: 100% ██████████
Class	Images	Labels	P	R	mAP@.5	mAP@
all	25	25	1	0.996	0.995	0.708

Epoch	gpu_mem	box	obj	cls	labels	img_size
96/99	2.02G	0.01692	0.005484	0	19	640: 100% ██████████
Class	Images	Labels	P	R	mAP@.5	mAP@
all	25	25	1	0.996	0.995	0.705

Epoch	gpu_mem	box	obj	cls	labels	img_size
97/99	2.02G	0.01678	0.005144	0	11	640: 100% ██████████
Class	Images	Labels	P	R	mAP@.5	mAP@
all	25	25	1	0.997	0.995	0.705

Epoch	gpu_mem	box	obj	cls	labels	img_size
98/99	2.02G	0.01691	0.006163	0	23	640: 100% ██████████
Class	Images	Labels	P	R	mAP@.5	mAP@
all	25	25	1	0.995	0.995	0.701

Epoch	gpu_mem	box	obj	cls	labels	img_size
99/99	2.02G	0.01649	0.005696	0	17	640: 100% ██████████
Class	Images	Labels	P	R	mAP@.5	mAP@
all	25	25	1	0.995	0.995	0.706

100 epochs completed in 0.289 hours.

Optimizer stripped from yolov5/runs/train/Model/weights/last.pt, 14.5MB

Optimizer stripped from yolov5/runs/train/Model/weights/best.pt, 14.5MB

Validating yolov5/runs/train/Model/weights/best.pt...

Fusing layers...

YOLOv5s summary: 213 layers, 7012822 parameters, 0 gradients

Class	Images	Labels	P	R	mAP@.5	mAP@
all	25	25	0.961	0.994	0.992	0.731

Results saved to **yolov5/runs/train/Model**

Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__ at 0x7fda7e0aab00>

Traceback (most recent call last):

File "/opt/conda/lib/python3.7/site-packages/torch/utils/data/dataloader.py", line 1358, in __del__

File "/opt/conda/lib/python3.7/site-packages/torch/utils/data/dataloader.py", line 1283, in _shutdown_workers

AttributeError: 'NoneType' object has no attribute 'python_exit_status'

Exception ignored in: <function _MultiProcessingDataLoaderIter.__del__ at 0x7fda7e0aab00>

Traceback (most recent call last):

File "/opt/conda/lib/python3.7/site-packages/torch/utils/data/dataloader.py", line 1358, in __del__

File "/opt/conda/lib/python3.7/site-packages/torch/utils/data/dataloader.py", line 1283, in _shutdown_workers

AttributeError: 'NoneType' object has no attribute 'python_exit_status'

When model trained, we need to save our it in order to use it at OCR in onnx format as below:

```
In[38]: !python ./yolov5/export.py --weight ./yolov5/runs/train/Model/weights/best.pt --include torchscript onnx
```

export: data=yolov5/data/coco128.yaml, weights=['./yolov5/runs/train/Model/weights/best.pt'], imgsz=[640, 640], batch_size=1, device=cpu, half=False, inplace=False, train=False, keras=False, optimize=False, int8=False, dynamic=False, simplify=False, opset=12, verbose=False, workspace=4, nms=False, agnostic_nms=False, topk_per_class=100, topk_all=100, iou_thresh=0.45, conf_thresh=0.25, include=['torchscript', 'onnx']

YOLOv5 🚀 v6.1-253-g75bbaa8 Python-3.7.12 torch-1.11.0 CPU

Fusing layers...

YOLOv5s summary: 213 layers, 7012822 parameters, 0 gradients

PyTorch: starting from yolov5/runs/train/Model/weights/best.pt with output shape (1, 25200, 6) (13.8 MB)

TorchScript: starting export with torch 1.11.0...

TorchScript: export success, saved as yolov5/runs/train/Model/weights/best.torchscript (27.2 MB)

requirements: onnx not found and is required by YOLOv5, attempting auto-update...

WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager. It is recommended to use a virtual environment instead: <https://pip.pypa.io/warnings/venv>

Collecting onnx

Downloading onnx-1.11.0-cp37-cp37m-manylinux_2_12_x86_64.manylinux2010_x86_64.whl (12.8 MB)

12.8/12.8 MB 36.7

MB/s eta 0:00:00

Requirement already satisfied: typing-extensions>=3.6.2.1 in /opt/conda/lib/python3.7/site-packages (from onnx) (4.2.0)

Requirement already satisfied: protobuf>=3.12.2 in /opt/conda/lib/python3.7/site-packages (from onnx) (3.20.1)

Requirement already satisfied: numpy>=1.16.6 in /opt/conda/lib/python3.7/site-packages (from onnx) (1.21.6)

Installing collected packages: onnx

Successfully installed onnx-1.11.0

requirements: 1 package updated per ['onnx']

requirements: ⚠ Restart runtime or rerun command for updates to take effect

ONNX: starting export with onnx 1.11.0...

ONNX: export success, saved as yolov5/runs/train/Model/weights/best.onnx (27.2 MB)

Export complete (18.15s)

Results saved to **/kaggle/working/yolov5/runs/train/Model/weights**

Detect: python detect.py --weights yolov5/runs/train/Model/weights/best.onnx

PyTorch Hub: model = torch.hub.load('ultralytics/yolov5', 'custom', 'yolov5/runs/train/Model/weights/best.onnx')

Validate: python val.py --weights yolov5/runs/train/Model/weights/best.onnx

Visualize: <https://netron.app>

Okay, after unzipping our model, we have effectively secured it. It is evident that a model folder exists. We are able to locate anticipated images and precisely identify the license plate. Even with a small percentage of incorrect results, our technology is still able to reliably identify the license plate. For me, the PR cow is ultimately crucial since it will reveal the detection rate. Our precision recall indicates that we can identify our model with 0.92 accuracy and 0.5 mean average precision. An example of a val_batch may be found below.

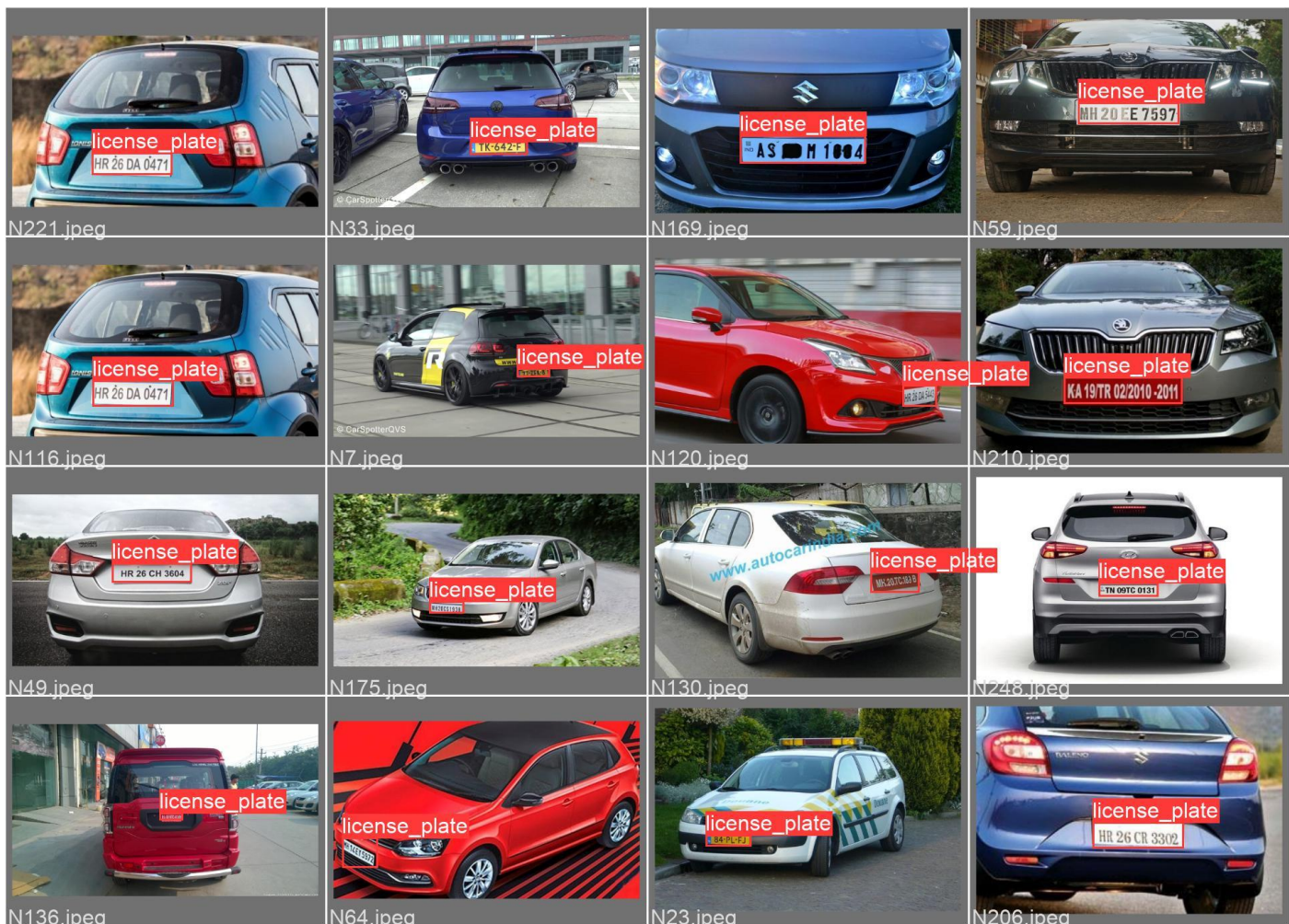


Figure 7.4 – Various Images of Real Time Number Plate Detection

7.4 PREDICTING RESULTS FROM YOLO

Let's do some images settings which is important to provide in model construction. Input settings we need to pass as 640 by 640.

In[39]:

settings

INPUT_WIDTH = 640

INPUT_HEIGHT = 640

In[40]:

LOAD THE IMAGE

img = io.imread('../input/number-plate-detection/TEST/TEST.jpeg')

fig = px.imshow(img)

fig.update_layout(width=700, height=400, margin=dict(l=10, r=10, b=10, t=10))

fig.update_xaxes(showticklabels=False).update_yaxes(showticklabels=False)

fig.show()



Figure 7.5 – Test Image

Now, we will load our YOLO model.

```
In[41]: # LOAD YOLO MODEL
net = cv2.dnn.readNetFromONNX('./yolov5/runs/train/Model/weights/best.onnx')
net.setPreferableBackend(cv2.dnn.DNN_BACKEND_OPENCV)
net.setPreferableTarget(cv2.dnn.DNN_TARGET_CPU)
```

I must now parse this image using my YOLO model in order to obtain the prediction. Important: We must set the size to 640 for the input width and 640 for the input height.

There are six columns and 25,200 rows. You may find information about the bounding box, which is center X, center Y, center W, and center H, in the first four columns. Also, 640 by 640 is used to standardize these data. The probability and confidence are the following two values. Since we only have one class—number plate—confidence indicates our level of confidence in recognizing the bounding box. The probability score of the class comes next.

Let's list the boxes and confidence and arrange them in a certain sequence. We also need to determine the image's X_factor and Y_factor.

Based on the likelihood and confidence scores, we must filter detections. Allow me to filter from class >0.25 using the confidence >0.4.

One of the main problems with YOLO modeling is that it will offer you several boxes for the same object, but it will also give you repeating boxes. We must execute the non-maximum suppression in order to do it. We must supply all of our clean boxes and confidences in order to complete it.

I'll use bounding boxes at the conclusion of that section of code to draw the rectangular box onto our image. and will put everything into functions.

```
In[42]: def get_detections(img,net):
# 1.CONVERT IMAGE TO YOLO FORMAT
image = img.copy()
```

```

row, col, d = image.shape

max_rc = max(row,col)
input_image = np.zeros((max_rc,max_rc,3),dtype=np.uint8)
input_image[0:row,0:col] = image

# 2. GET PREDICTION FROM YOLO MODEL
blob = cv2.dnn.blobFromImage(input_image,1/255,(INPUT_WIDTH,INPUT_HEIGHT),swapRB=True,crop=False)
net.setInput(blob)
preds = net.forward()
detections = preds[0]

return input_image, detections

def non_maximum_supression(input_image,detections):

# 3. FILTER DETECTIONS BASED ON CONFIDENCE AND PROBABILITY SCORE

# center x, center y, w , h, conf, proba
boxes = []
confidences = []

image_w, image_h = input_image.shape[:2]
x_factor = image_w/INPUT_WIDTH
y_factor = image_h/INPUT_HEIGHT

for i in range(len(detections)):
    row = detections[i]
    confidence = row[4] # confidence of detecting license plate
    if confidence > 0.4:
        class_score = row[5] # probability score of license plate
        if class_score > 0.25:
            cx, cy , w, h = row[0:4]

            left = int((cx - 0.5*w)*x_factor)
            top = int((cy-0.5*h)*y_factor)
            width = int(w*x_factor)
            height = int(h*y_factor)
            box = np.array([left,top,width,height])

            confidences.append(confidence)
            boxes.append(box)

# 4.1 CLEAN
boxes_np = np.array(boxes).tolist()
confidences_np = np.array(confidences).tolist()

# 4.2 NMS
index = cv2.dnn.NMSBoxes(boxes_np,confidences_np,0.25,0.45)

return boxes_np, confidences_np, index

def drawings(image,boxes_np,confidences_np,index):
# 5. Drawings
for ind in index:
    x,y,w,h = boxes_np[ind]
    bb_conf = confidences_np[ind]

```



```

conf_text = 'plate: {:.0f}%'.format(bb_conf*100)
license_text = extract_text(image,boxes_np[ind])

cv2.rectangle(image,(x,y),(x+w,y+h),(255,0,255),2)
cv2.rectangle(image,(x,y-30),(x+w,y),(255,0,255),-1)
cv2.rectangle(image,(x,y+h),(x+w,y+h+25),(0,0,0),-1)

cv2.putText(image,conf_text,(x,y-10),cv2.FONT_HERSHEY_SIMPLEX,0.7,(255,255,255),1)
cv2.putText(image,license_text,(x,y+h+27),cv2.FONT_HERSHEY_SIMPLEX,0.7,(0,255,0),1)

return image

```

```

In[43]: # predictions flow with return result
def yolo_predictions(img,net):
    # step-1: detections
    input_image, detections = get_detections(img,net)
    # step-2: NMS
    boxes_np, confidences_np, index = non_maximum_supression(input_image, detections)
    # step-3: Drawings
    result_img = drawings(img,boxes_np,confidences_np,index)
    return result_img

```

Here, in addition to detecting license plates, we will also extract tests from them. Let's do it once more as we have already done it. We're going to write a function. We must first grab our x, y, w, and h from the bounding box and extract ROI. I rank it as the most important function in our drawing par

```

In[44]: # extrating text
def extract_text(image,bbox):
    x,y,w,h = bbox
    roi = image[y:y+h, x:x+w]

    if 0 in roi.shape:
        return 'no number'

    else:
        text = pt.image_to_string(roi)
        text = text.strip()

    return text

```

```

In[45]: # test
img = io.imread('../input/number-plate-detection/TEST/TEST.jpeg')
results = yolo_predictions(img,net)

```

```

In[46]: fig = px.imshow(img)
fig.update_layout(width=700, height=400, margin=dict(l=10, r=10, b=10, t=10))
fig.update_xaxes(showticklabels=False).update_yaxes(showticklabels=False)
fig.show()

```



Figure 7.6 – Detecting Number Plate

The model's accuracy is actually fairly precise. Even with the small number of misclassifications, our model is still operating quite effectively. Increasing the amount of data you send in will allow you to predict very good outcomes and achieve ever-higher levels of accuracy.

7.5 REAL TIME OBJECT DETECTION

In real time prediction my point is that the detection is very perfect and in order to get very, very good predictions, we need to get very high 4K definition camera so that we can able to extract the text properly.

```
cap = cv2.VideoCapture('../input/number-plate-detection/TEST/TEST.mp4')
while True:
    ret, frame = cap.read()

    if ret == False:
        print('Unable to read video')
        break

    results = yolo_predictions(frame,net)

    cv2.namedWindow('YOLO',cv2.WINDOW_KEEPRATIO)
    cv2.imshow('YOLO',results)
    if cv2.waitKey(30) == 27 :
        break

cv2.destroyAllWindows()
cap.release()
```



Figure 7.7 – Real Time Number Plate Detection Using YoloV5

8. CONCLUSION

Vehicle tracking has grown in importance as a study topic due to the rise in automobiles, which helps with effective traffic management, surveillance, and the recovery of stolen vehicles. Efficient real-time license plate detection and recognition are crucial for this reason. License plate recognition is a major difficulty in developing countries because of variations in backdrop and text color, font style, license plate size, and non-standard characters. This work uses a deep learning approach to increase the effectiveness of license plate identification in order to get around such problems. The gathered photos have been taken at several angles of rotation, at different distances from the camera, and with different lighting and contrast settings. They have also been verified to yield a high identification rate. Law enforcement authorities and private entities can enhance homeland security by using this technique effectively. The existing algorithm may be trained and validated using the hybrid classifier technique in future work, and the robustness of the license plate recognition system in variable weather circumstances may be enhanced.

9. REFERENCES

1. Asikpalysik. (n.d.-a). ASIKPALYSIK/automatic-license-plate-detection: In this project we utilize opencv T in order to identify the license number plates and the python pytesseract for the characters and digits extraction from the plate. as well this project will presents a robust and efficient ALPR system based on the state-of-the-art yolo object detector. we build web app with a python program that automatically recognizes the license number plate by the end of this Journi. the results have shown that the trained neural network is able to perform with high accuracy of nearly 90-95 percent in recognizing license plates in low resolution images using this system. GitHub. <https://github.com/Asikpalysik/Automatic-License-Plate-Detection>
2. Asikpalysik. (n.d.-c). Automatic-license-plate-detection/images at main · ASIKPALYSIK/automatic-license-plate-detection. GitHub. <https://github.com/Asikpalysik/Automatic-License-Plate-Detection/tree/main/images>
3. Google. (n.d.). Google. <https://www.google.co.in/>
4. HumanSignal. (n.d.). Labellmg/readme/readme.zh.rst at master · HumanSignal/labellmg. GitHub. <https://github.com/HumanSignal/labellmg/blob/master/readme/README.zh.rst>
5. Home. OpenCV. (2024, July 10). <https://opencv.org/>
6. HumanSignal. (n.d.). Labellmg/readme/readme.zh.rst at master · HumanSignal/labellmg. GitHub. <https://github.com/HumanSignal/labellmg/blob/master/readme/README.zh.rst>
7. Introducing chatgpt - openai. (n.d.). <https://openai.com/index/chatgpt>
8. Pytesseract. PyPI. (n.d.). <https://pypi.org/project/pytesseract/>
9. Tensorflow. TensorFlow. (n.d.). <https://www.tensorflow.org/>
10. Tesseract-Ocr. (n.d.). Tesseract-OCR/tesseract: Tesseract open source OCR engine (main repository). GitHub. <https://github.com/tesseract-ocr/tesseract>
11. Ultralytics. (n.d.). Ultralytics/yolov5: Yolov5 🚀 in PyTorch > ONNX > CoreML > TFLite. GitHub. <https://github.com/ultralytics/yolov5>